

Lecture Notes on Software Engineering

Unit-1

INTRODUCTION TO SOFTWARE ENGINEERING

The term **software engineering** is composed of two words, software and engineering.

Software is more than just a program code. A program is an executable code, which serves some computational purpose. Software is considered to be a collection of executable programming code, associated libraries and documentations. Software, when made for a specific requirement is called **software product**.

Engineering on the other hand, is all about developing products, using well-defined, scientific principles and methods.

So, we can define **software engineering** as an engineering branch associated with the development of software product using well-defined scientific principles, methods and procedures. The outcome of software engineering is an efficient and reliable software product.

IEEE defines software engineering as:

The application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software.

We can alternatively view it as a systematic collection of past experience. The experience is arranged in the form of methodologies and guidelines. A small program can be written without using software engineering principles. But if one wants to develop a large software product, then software engineering principles are absolutely necessary to achieve a good quality software cost effectively.

Without using software engineering principles it would be difficult to develop large programs. In industry it is usually needed to develop large programs to accommodate multiple functions. A problem with developing such large commercial programs is that the complexity and difficulty levels of the programs increase exponentially with their sizes.

Software engineering helps to reduce this programming complexity. **Software engineering principles use two important techniques to reduce problem complexity: abstraction and decomposition.** The principle of abstraction implies that a problem can be simplified by omitting irrelevant details. In other words, the main purpose of abstraction is to consider only those aspects of the problem that are relevant for certain purpose and suppress other aspects that are not relevant for the given purpose. Once the simpler problem is solved, then the omitted details can be taken into consideration to solve the next lower level abstraction, and so on. Abstraction is a powerful way of reducing the complexity of the problem. The other approach to tackle problem complexity is decomposition. In this technique, a complex problem is divided into several smaller problems and then the smaller

problems are solved one by one. However, in this technique any random decomposition of a problem into smaller parts will not help. The problem has to be decomposed such that each component of the decomposed problem can be solved independently and then the solution of the different components can be combined to get the full solution. A good decomposition of a problem should minimize interactions among various components. If the different subcomponents are interrelated, then the different components cannot be solved separately and the desired reduction in complexity will not be realized.

NEED OF SOFTWARE ENGINEERING

The need of software engineering arises because of higher rate of change in user requirements and environment on which the software is working.

- **Large software** - It is easier to build a wall than to a house or building, likewise, as the size of software become large engineering has to step to give it a scientific process.
- **Scalability**- If the software process were not based on scientific and engineering concepts, it would be easier to re-create new software than to scale an existing one.
- **Cost**- As hardware industry has shown its skills and huge manufacturing has lower down the price of computer and electronic hardware. But the cost of software remains high if proper process is not adapted.
- **Dynamic Nature**- The always growing and adapting nature of software hugely depends upon the environment in which the user works. If the nature of software is always changing, new enhancements need to be done in the existing one. This is where software engineering plays a good role.
- **Quality Management**- Better process of software development provides better and quality software product.

CHARACTERISTICS OF GOOD SOFTWARE

A software product can be judged by what it offers and how well it can be used. This software must satisfy on the following grounds:

- Operational
- Transitional
- Maintenance

Well-engineered and crafted software is expected to have the following characteristics:

Operational

This tells us how well software works in operations. It can be measured on:

- Budget
- Usability
- Efficiency
- Correctness
- Functionality
- Dependability
- Security
- Safety

Transitional

This aspect is important when the software is moved from one platform to another:

- Portability
- Interoperability
- Reusability
- Adaptability

Maintenance

This aspect briefs about how well software has the capabilities to maintain itself in the ever-changing environment:

- Modularity
- Maintainability

- Flexibility
- Scalability

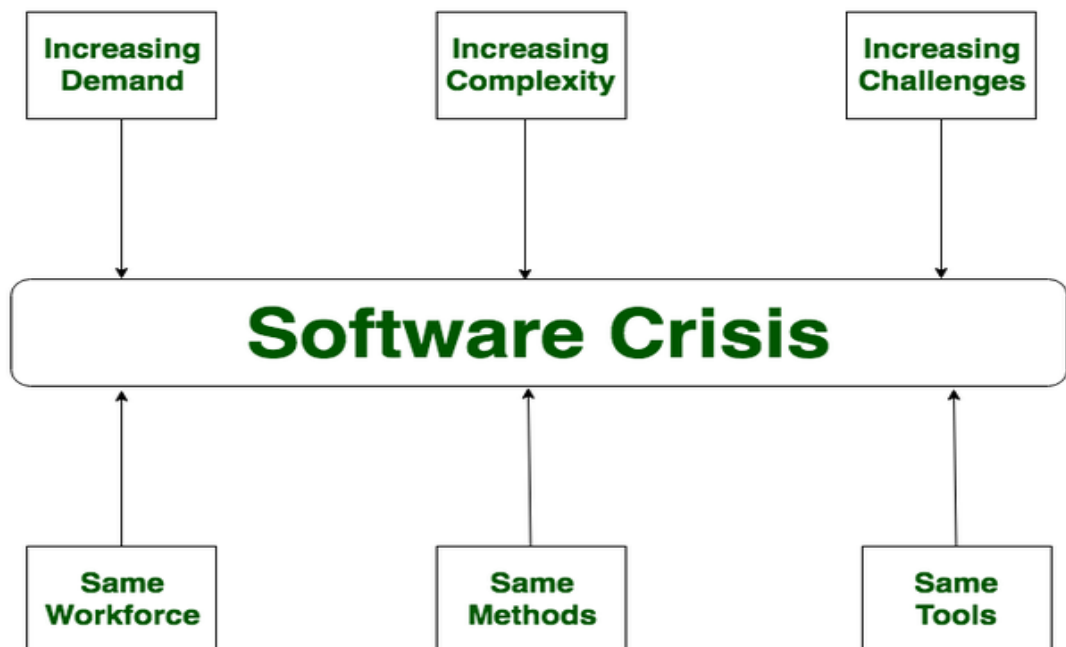
In short, Software engineering is a branch of computer science, which uses well-defined engineering concepts required to produce efficient, durable, scalable, in-budget and on-time software products.

“Software crisis”

The term “software crisis” refers to the numerous challenges and difficulties faced by the software industry during the 1960s and 1970s.

It became clear that old methods of developing software couldn’t keep up with the growing complexity and demands of new projects. This led to high costs, delays, and poor-quality software.

New methodologies and tools were needed to address these issues.



Software Crisis is a term used in computer science for the difficulty of writing useful and efficient computer programs in the required time. The software crisis was due to using the

same workforce, same methods, and same tools even though rapidly increasing software demand, the complexity of software, and software challenges. With the increase in software complexity, many software problems arose because existing methods were insufficient.

Causes of Software Crisis

Following are the causes of Software Crisis:

The cost of owning and maintaining software was as expensive as developing the software.

At that time Projects were running overtime.

At that time Software was very inefficient.

The quality of the software was low quality.

Software often did not meet user requirements.

At that time Software was never delivered.

Non-optimal resource utilization.

Challenging to alter, debug, and enhance.

The software complexity is harder to change.

Factors Contributing to Software Crisis

Factor Contributing to Software Crisis are:

Poor project management.

Lack of adequate training in software engineering.

Less skilled project members.

Low productivity improvements.

Solution of Software Crisis:

There is no single solution to the crisis. One possible solution to a software crisis is *Software Engineering* because **software engineering is a systematic, disciplined, and quantifiable approach. For preventing software crises, there are some guidelines:**

Reduction in software over budget.

The quality of the software must be high.

Less time is needed for a software project.

Experienced and skilled people working on the software project.

Software must be delivered.

Software must meet user requirement.

SOFTWARE DEVELOPMENT LIFE CYCLE

Software Development Life Cycle (SDLC) is a well-defined, structured sequence of stages in software engineering to develop the intended software product.

SDLC ACTIVITIES

SDLC provides a series of steps to be followed to design and develop a software product efficiently. SDLC framework includes the following steps:



Fig 1: SDLC framework

Communication

This is the first step where the user initiates the request for a desired software product. He contacts the service provider and tries to negotiate the terms. He submits his request to the service providing organization in writing.

Requirement Gathering

This step onwards the software development team works to carry out the project. The team holds discussions with various stakeholders from problem domain and tries to bring out as much information as possible on their requirements. The requirements are contemplated and segregated into user requirements, system requirements and functional requirements. The requirements are collected using a number of practices as given -

- studying the existing or obsolete system and software
- conducting interviews of users and developers
- referring to the database
- Collecting answers from the questionnaires.

Feasibility Study

After requirement gathering, the team comes up with a rough plan of software process. At this step the team analyzes if the software can be made to fulfill all requirements of the user and if there is any possibility of the software being no more useful. If the project is financially, practically and technologically feasible for the organization to take up, they take up the project else they let it go.

System Analysis

At this step the developers decide a roadmap of their plan and try to bring up the best software model suitable for the project. System analysis includes understanding of software product limitations, learning system related problems or changes to be done in existing systems beforehand, identifying and addressing the impact of project on organization and personnel etc. The project team analyzes the scope of the project and plans the schedule and resources accordingly.

Software Design

Next step is to bring down whole knowledge of requirements and analysis on the desk and design the software product. The inputs from users and information gathered in requirement gathering phase are the inputs of this step. The output of this step comes in the form of two

designs; logical design and physical design. Engineers produce meta-data and data dictionaries, logical diagrams, data-flow diagrams and in some cases pseudo codes.

Coding

This step is also known as programming phase. The implementation of software design starts in terms of writing program code in the suitable programming language and developing error-free executable programs efficiently.

Testing

An estimate says that 50% of whole software development process should be tested. Errors may ruin the software from critical level to its own removal. Software testing is done while coding by the developers and thorough testing is conducted by testing experts at various levels of code such as module testing, program testing, product testing, in-house testing and testing the product at user's end. Early discovery of errors and their remedy is the key to reliable software.

Integration

Software may need to be integrated with the libraries, databases and other program(s). This stage of SDLC is involved in the integration of software with outer world entities.

Implementation

This means installing the software on user machines. At times, software needs post-installation configurations at user end. Software is tested for portability and adaptability and integration related issues are solved during implementation.

Operation and Maintenance

This phase confirms the software operation in terms of more efficiency and less errors. If required, the users are trained on, or aided with the documentation on how to operate the software and how to keep the software operational. The software is maintained timely by updating the code according to the changes taking place in user end environment or technology. This phase may face challenges from hidden bugs and real-world unidentified problems.

Disposition

As time elapses, the software may decline on the performance front. It may go completely obsolete or may need intense up gradation. Hence a pressing need to eliminate a major portion of the system arises. This phase includes archiving data and required software components, closing down the system, planning disposition activity and terminating system at appropriate end-of-system time.

THE NEED FOR A SOFTWARE LIFE CYCLE MODEL

The development team must identify a suitable life cycle model for the particular project and then adhere to it. Without using of a particular life cycle model the development of a software product would not be in a systematic and disciplined manner. When a software product is being developed by a team there must be a clear understanding among team members about when and what to do. Otherwise it would lead to chaos and project failure. This problem can be illustrated by using an example. Suppose a software development problem is divided into several parts and the parts are assigned to the team members. From then on, suppose the team members are allowed the freedom to develop the parts assigned to them in whatever way they like. It is possible that one member might start writing the code for his part, another might decide to prepare the test documents first, and some other engineer might begin with the design phase of the parts assigned to him. This would be one of the perfect recipes for project failure. A software life cycle model defines entry and exit criteria for every phase. A phase can start only if its phase-entry criteria have been satisfied. So without software life cycle model the entry and exit criteria for a phase cannot be recognized. Without software life cycle models it becomes difficult for software project managers to monitor the progress of the project.

SOFTWARE DEVELOPMENT PARADIGM

The software development paradigm helps developer to select a strategy to develop the software. A software development paradigm has its own set of tools, methods and procedures, which are expressed clearly and defines software development life cycle. A few of software development paradigms or process models are defined as follows:

CLASSICAL WATERFALL MODEL

Classical waterfall model is the simplest model of software development paradigm. It says the all the phases of SDLC will function one after another in linear manner. That is, when the first phase is finished then only the second phase will start and so on.

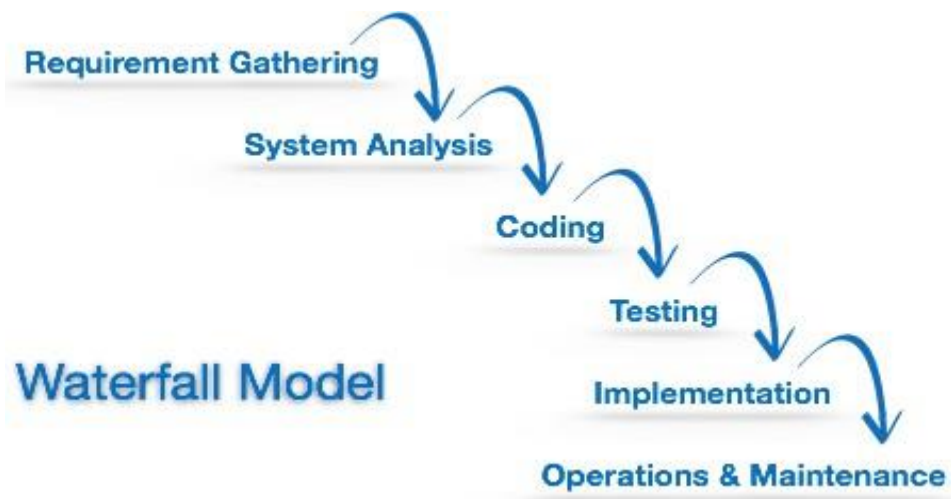


Fig 2: Waterfall Model

This model assumes that everything is carried out and taken place perfectly as planned in the previous stage and there is no need to think about the past issues that may arise in the next phase. This model does not work smoothly if there are some issues left at the previous step. The sequential nature of model does not allow us go back and undo or redo our actions.

Waterfall Model

The **Waterfall Model** is one of the earliest and most traditional software development methodologies, widely known for its linear and sequential approach to software development. It is based on the idea of moving through distinct phases, where each phase must be completed before the next one begins. The model is often compared to a waterfall due to its structured and step-by-step nature.

Phases of the Waterfall Model

Requirement Analysis:

In this phase, the requirements of the system are gathered and documented in detail. This is typically done by engaging with stakeholders, customers, and end-users to understand what they expect from the software system.

The goal is to create a **Software Requirements Specification (SRS)** document that serves as a blueprint for the system.

Key Activities:

Identifying user needs.

Defining functional and non-functional requirements.

Gathering business requirements and constraints.

System Design:

Once the requirements are clear, the system design phase begins. This phase is where the architectural design of the software is created, including both high-level design and low-level design.

High-Level Design (HLD): Defines the overall structure of the system, including the system architecture, modules, data flow, and external interfaces.

Low-Level Design (LLD): Involves detailed design specifications for each module,

including data structures, algorithms, and other technical aspects.

Key Activities:

Creating system architecture.

Designing the software components and database schema.

Creating detailed technical specifications.

Implementation (Coding):

After the design phase, the development or coding phase begins. During this phase, the system is actually built by the development team based on the design documents.

Developers write code for each module or component, and integrate them into a functioning system.

Key Activities:

Writing the software code.

Unit testing the code.

Ensuring the code aligns with the design specifications.

Integration and Testing:

After coding is completed, the system is tested to ensure that all components work together as expected. This phase involves various levels of testing, including unit testing, integration testing, system testing, and acceptance testing.

The goal is to identify defects, verify that the system meets the requirements, and ensure that it performs as expected.

Key Activities:

Conducting functional testing (e.g., does the software do what it was designed to do?).

Conducting non-functional testing (e.g., performance, security, usability).

Identifying and fixing defects.

Deployment:

Once the system is tested and approved, it is ready for deployment. The system is released to the users or transferred to the production environment for actual use.

In some cases, deployment is done in stages, starting with a limited user group before full deployment.

Key Activities:

Preparing deployment documentation.

Installing the system in the production environment.

Conducting any final user training or support.

Maintenance:

After the system is deployed, the maintenance phase begins. This involves ongoing updates and enhancements to the system as new issues arise, or additional features are requested by users.

Maintenance also involves addressing any defects found post-deployment, applying security patches, and ensuring continued system performance.

Key Activities:

Fixing bugs and defects.

Adding new features or enhancements.

Performing software updates to ensure compatibility with new systems.

Advantages of the Waterfall Model

Simple and Easy to Understand:

The Waterfall Model is straightforward, with clearly defined phases. It is easy to understand and communicate to both technical and non-technical stakeholders.

Structured Approach:

The model's linear approach helps to structure the development process clearly, reducing ambiguity about what happens in each phase.

Well-Defined Phases:

Each phase has specific deliverables, which makes it easy to track progress and manage the project.

Easy to Manage:

Because of its sequential nature, it is easier to manage the development process, as each phase must be completed before moving to the next. This makes it suitable for smaller, less complex projects with well-defined requirements.

Documentation-Heavy:

Since each phase has clear documentation and deliverables, it makes the process more transparent and helps maintain a record for future reference.

Disadvantages of the Waterfall Model

Inflexibility:

Once a phase is completed, going back to a previous phase is difficult and costly. This makes the model rigid and unsuitable for projects where requirements are expected to change during development.

Late Testing:

Testing is done only after the system is fully developed, meaning defects may not be discovered until later in the process. This can make fixing issues more expensive and time-consuming.

Assumes Complete Requirements from the Start:

The model assumes that all requirements can be defined upfront and are unlikely to change, which is not always the case in real-world software development projects.

Not Ideal for Complex or Large Projects:

The Waterfall Model is not well-suited for complex, large-scale projects where requirements and solutions are expected to evolve over time. It lacks flexibility to handle changes effectively.

Late User Feedback:

Since the system is not tested or delivered until the end of the process, user feedback comes

very late. This may lead to dissatisfaction if the system does not meet the user's expectations or needs.

When to Use the Waterfall Model

Clear and Fixed Requirements:

When the software requirements are well-understood, stable, and unlikely to change throughout the development process.

Small-Scale Projects:

For smaller projects with straightforward, well-defined requirements where there is little need for iteration or flexibility.

Regulated Environments:

For projects in regulated environments where formal documentation and detailed plans are essential (e.g., healthcare, aerospace).

Projects with Well-Defined Outcomes:

Projects where the outcome is well-understood and there are clear goals and deliverables, making the linear flow of the Waterfall Model a good fit.

Note:

The **Waterfall Model** is a traditional, linear approach to software development that is suitable for smaller projects with clear and stable requirements. However, its rigidity, late testing, and difficulty in handling changes make it less suitable for large, complex, or dynamic projects. Despite its limitations, the Waterfall Model's simplicity, structured approach, and well-defined phases continue to make it valuable in certain contexts, especially when working with regulated industries or well-understood requirements.

This model is best suited when developers already have designed and developed similar software in the past and is aware of all its domains.

ITERATIVE MODEL

This model leads the software development process in iterations. It projects the process of development in cyclic manner repeating every step after every cycle of SDLC process.

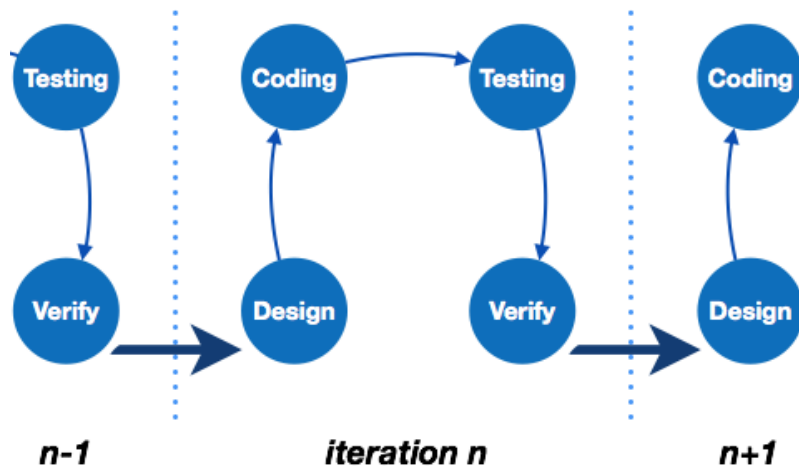


Fig 3: Iterative Model

The software is first developed on very small scale and all the steps are followed which are taken into consideration. Then, on every next iteration, more features and modules are designed, coded, tested and added to the software. Every cycle produces a software, which is complete in itself and has more features and capabilities than that of the previous one.

After each iteration, the management team can do work on risk management and prepare for the next iteration. Because a cycle includes small portion of whole software process, it is easier to manage the development process but it consumes more resources.

PROTOTYPING MODEL

A prototype is a toy implementation of the system. A prototype usually exhibits limited functional capabilities, low reliability, and inefficient performance compared to the actual software. A prototype is usually built using several shortcuts. The shortcuts might involve using inefficient, inaccurate, or dummy functions. The shortcut implementation of a function, for example, may produce the desired results by using a table look-up instead of performing the actual computations. A prototype usually turns out to be a very crude version of the actual system.

Following is the stepwise approach to design a software prototype:

1. **Basic Requirement Identification:** This step involves understanding the very basic product requirements especially in terms of user interface. The more intricate details of the internal design and external aspects like performance and security can be ignored at this stage.
2. **Developing the initial Prototype:** The initial Prototype is developed in this stage, where the very basic requirements are showcased and user interfaces are provided. These features may not exactly work in the same manner internally in the actual software developed and the workarounds are used to give the same look and feel to the customer in the prototype developed.
3. **Review of the Prototype:** The prototype developed is then presented to the customer and the other important stakeholders in the project. The feedback is collected in an organized manner and used for further enhancements in the product under development.
4. **Revise and enhance the Prototype:** The feedback and the review comments are discussed during this stage and some negotiations happen with the customer based on factors like, time and budget constraints and technical feasibility of actual implementation. The changes accepted are again incorporated in the new Prototype developed and the cycle repeats until customer expectations are met.

Key Characteristics of the Prototyping Model

Incremental Refinement: The prototype is developed in iterations. Each version of the prototype incorporates user feedback and evolves toward the final product.

User Feedback: Since users interact with the prototype early on, the feedback they provide is crucial for improving the software.

Rapid Development: The prototype is typically developed quickly with minimal functionality to give the users a tangible representation of what the final system might look like.

Flexible and Adaptive: The Prototyping Model is flexible because it allows for changes based on feedback at various stages of development.

Advantages of the Prototyping Model

Improved User Involvement:

Users are actively involved throughout the development process, which ensures that their needs and expectations are addressed early and continuously.

Faster Feedback:

Since a prototype is created quickly, users can provide feedback early in the development process, allowing for adjustments before the final product is completed.

Better Understanding of Requirements:

It helps clarify requirements that may be unclear or difficult to define initially. As users interact with the prototype, they gain a better understanding of their needs, which helps refine the software.

Reduced Risk of Misunderstandings:

The iterative nature of prototyping helps identify and resolve misunderstandings between developers and users early in the process, reducing the risk of delivering an unwanted product.

Flexibility:

The Prototyping Model allows for flexibility in changing requirements, as the development is iterative, and new features or changes can be easily incorporated.

User Satisfaction:

Since users can see the progress and interact with the software early, they are more likely to be satisfied with the final product.

Disadvantages of the Prototyping Model

Incomplete Requirements:

The initial requirements may not be fully detailed, which can lead to incomplete or vague specifications. This might cause confusion or missed functionality during the development process.

User Confusion:

Users might mistake the prototype for the final product. Since the prototype is usually not fully functional or polished, this can lead to unrealistic expectations.

Cost of Iteration:

Iterative development can become expensive, especially if many cycles are needed to refine the prototype. If the process continues for too many iterations, it can increase costs and delay the project.

Potential for Scope Creep:

Due to the evolving nature of prototypes, there is a risk of adding unnecessary features or changes as new feedback comes in, leading to scope creep and delays.

Lack of Documentation:

Prototypes are often developed quickly and may lack the level of documentation that a fully developed system would have. This can lead to challenges when transitioning from the prototype to the final system.

Unclear Final System Design:

As the system evolves based on user feedback, the final design may not be as well-defined as it would be in other methodologies, leading to integration or architectural challenges.

When to Use the Prototyping Model

The Prototyping Model is particularly useful in the following scenarios:

When Requirements Are Not Fully Known:

If the full scope and requirements of the system are unclear at the beginning of the project, prototyping helps clarify those requirements through user feedback.

When User Feedback is Crucial:

If user input is essential to the success of the project, the Prototyping Model enables continuous interaction and improvement based on user feedback.

For Projects with Changing Requirements:

Prototyping works well in dynamic environments where requirements are expected to evolve or change throughout the development process.

For Small to Medium-Sized Projects:

Prototyping can be more effective for smaller, less complex projects, as it allows for quick feedback and iteration without overly complicating the development process.

Note:

The Prototyping Model offers a flexible and user-centered approach to software development. By creating early prototypes, gathering feedback, and refining the system iteratively, this model ensures that the final product better aligns with user expectations.

While it is advantageous for clarifying requirements and improving user satisfaction, it requires careful management to avoid scope creep, cost overruns, and user confusion.

The Prototyping Model is best suited for projects where requirements are not fully understood initially or are likely to change over time.

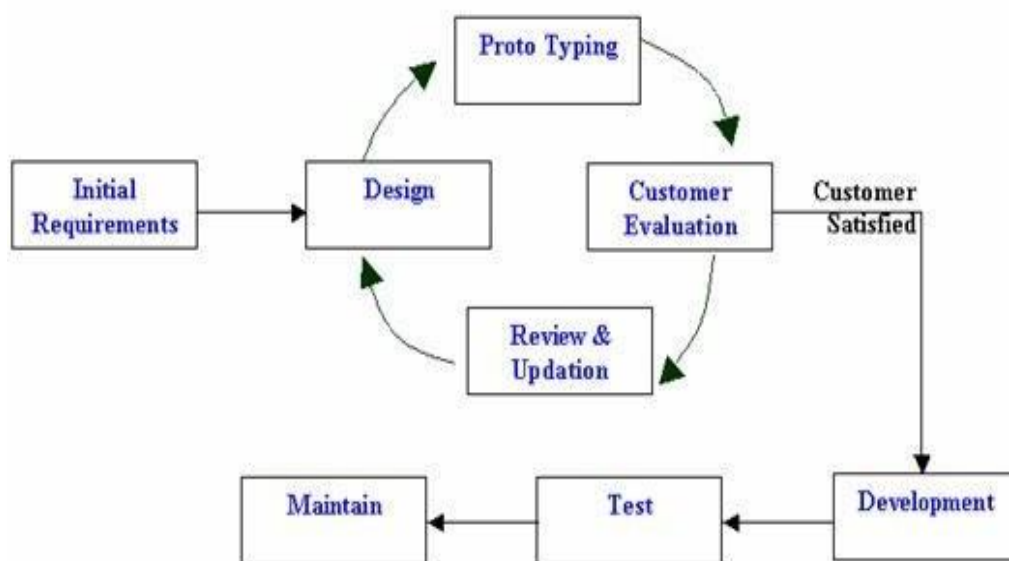


Fig 4: Prototype Model

SPIRAL MODEL

The Spiral model of software development is shown in fig.5. The diagrammatic representation of this model appears like a spiral with many loops. The exact number of loops in the spiral is not fixed. Each loop of the spiral represents a phase of the software process. For example, the innermost loop might be concerned with feasibility study. The next loop with requirements specification, the next one with design, and so on. Each phase in this model is split into four sectors (or quadrants) as shown in the fig.5. The following activities are carried out during each phase of a spiral model.

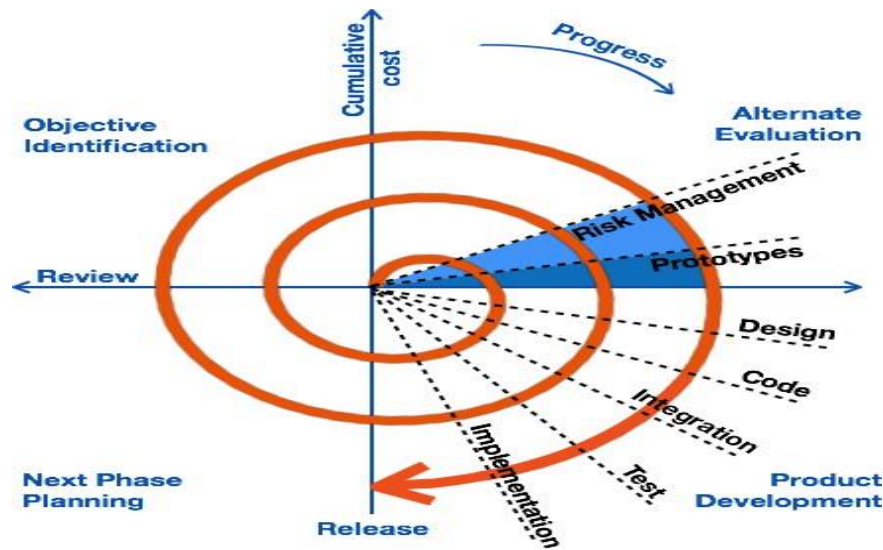


Fig 5: Spiral Model

- First quadrant (Objective Setting)

- During the first quadrant, it is needed to identify the objectives of the phase.
- Examine the risks associated with these objectives.

- Second Quadrant (Risk Assessment and Reduction)

- A detailed analysis is carried out for each identified project risk.
- Steps are taken to reduce the risks. For example, if there is a risk that the requirements are inappropriate, a prototype system may be developed.

- Third Quadrant (Development and Validation)

- Develop and validate the next level of the product after resolving the identified risks.

- Fourth Quadrant (Review and Planning)

- Review the results achieved so far with the customer and plan the next iteration around the spiral.
- Progressively more complete version of the software gets built with each iteration around the spiral.

The **Spiral Model** is a software development process model that combines elements of both iterative development and the traditional Waterfall model. It was introduced by **Barry Boehm** in 1986 and focuses on risk management through continuous refinement of the software through iterative cycles or "spirals."

Here's a breakdown of the key features and phases of the Spiral Model:

1. Phases of the Spiral Model:

The model is divided into distinct phases, which are repeated iteratively:

Phase 1: Planning

Objective: Involves defining the objectives, scope, and constraints of the project.

Activities:

Identifying project requirements

Conducting feasibility analysis (technical, operational, and financial)

Setting goals and deliverables

Phase 2: Risk Analysis

Objective: Identify and evaluate potential risks that could impact the project.

Activities:

Identifying risks (technical, financial, schedule-related, etc.)

Developing strategies to mitigate or resolve risks

Prototyping if necessary to reduce uncertainty

Phase 3: Engineering and Development

Objective: This phase involves the actual development of the system, where requirements are designed, built, and tested.

Activities:

Detailed design, coding, and testing of components or features

Building and verifying the prototype (if applicable)

Integration of components

Phase 4: Evaluation and Review

Objective: Evaluate the progress and quality of the project to determine if it meets requirements.

Activities:

Gathering feedback from stakeholders

Assessing the system's progress against objectives

Planning the next cycle or iteration based on feedback

2. Spiral Cycles:

The process is performed in a series of **spirals**, each of which involves revisiting the same phases with increasingly refined details.

Each spiral cycle results in a version of the software product. This allows for continuous

assessment, risk management, and refinement.

The number of spirals required depends on the complexity of the project.

3. Risk Management Focus:

One of the main benefits of the Spiral Model is its emphasis on **continuous risk assessment** throughout the project.

At the start of each cycle, risks are reassessed and, where necessary, mitigated. This makes the model suitable for large and complex projects where risks can evolve over time.

4. Iterative and Incremental:

Unlike the Waterfall model, which follows a strict linear approach, the Spiral Model is both iterative and incremental. It allows for revisiting previous phases and refining the product iteratively, rather than proceeding through each phase just once.

New versions of the software are developed incrementally, with each iteration incorporating more functionality and refinement.

Advantages of the Spiral Model:

Risk Reduction: It emphasizes risk assessment and mitigation, which makes it a good choice for large or complex projects.

Flexibility: Allows for changes to requirements, design, and goals as the project progresses.

Early Prototyping: The model allows for early prototyping, which helps in visualizing the system early in the process and improving user feedback.

Disadvantages of the Spiral Model:

Complexity: It can be difficult to manage and implement due to its iterative nature and the need for careful risk management.

Costly: Because it involves repeating phases, it can be resource-intensive, especially in the early stages.

Time-Consuming: The risk analysis and review steps may require significant time and effort for each iteration.

Use Cases:

The Spiral Model is most suitable for:

Large, complex, and high-risk projects (such as large enterprise systems, defense, or research

projects).

Projects that are prone to frequent changes and require constant feedback.

Agile Development Models:

In earlier days, the Iterative Waterfall Model was very popular for completing a project. But nowadays, developers face various problems while using it to develop software. The main difficulties included handling customer change requests during project development and the high cost and time required to incorporate these changes. To overcome these drawbacks of the Waterfall Model, in the mid-1990s the Agile Software Development model was proposed.

The Agile Model was primarily designed to help a project adapt quickly to change requests. So, the main aim of the Agile model is to facilitate quick project completion.

To accomplish this task, agility is required. Agility is achieved by fitting the process to the project and removing activities that may not be essential for a specific project. Also, anything that is a waste of time and effort is avoided.

Steps in the Agile Model

The agile model is a combination of iterative and incremental process models. The steps involve in agile SDLC models are:

Requirement gathering

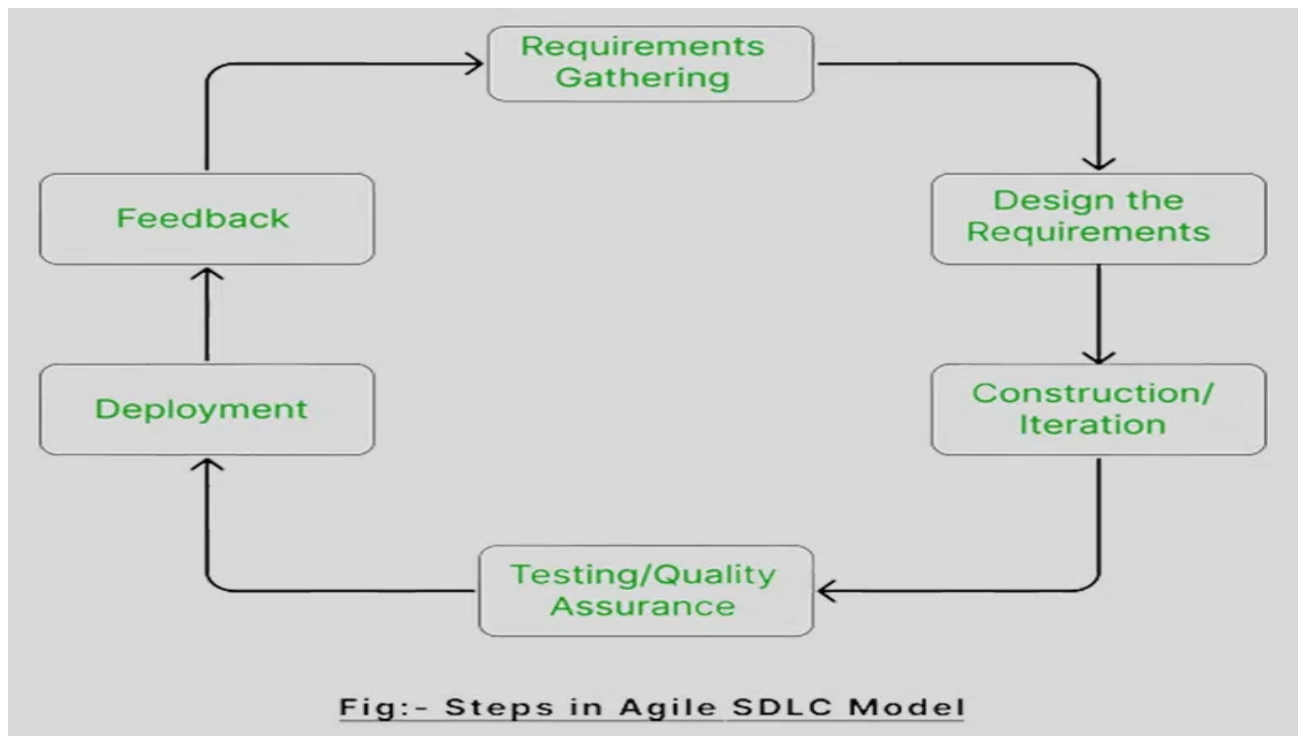
Design the Requirements

Construction / Iteration

Testing / Quality Assurance

Deployment

Feedback



Requirement Gathering:- In this step, the development team must gather the requirements, by interaction with the customer. Development team should plan the time and effort needed to build the project. Based on this information you can evaluate technical and economical feasibility.

Design the Requirements:- In this step, the development team will use user-flow-diagram or high-level UML diagrams to show the working of the new features and show how they will apply to the existing software. Wire framing and designing user interfaces are done in this phase.

Construction / Iteration:- In this step, development team members start working on their project, which aims to deploy a working product.

Testing / Quality Assurance:- Testing involves Unit Testing, Integration Testing, and System Testing. A brief introduction of these three tests is as follows:

Unit Testing:- Unit testing is the process of checking small pieces of code to ensure that the individual parts of a program work properly on their own. Unit testing is used to test individual blocks (units) of code.

Integration Testing:- Integration testing is used to identify and resolve any issues that may arise when different units of the software are combined.

System Testing:- Goal is to ensure that the software meets the requirements of the users and that it works correctly in all possible scenarios.

Deployment:- In this step, the development team will deploy the working project to end users.

Feedback:- This is the last step of the Agile Model. In this, the team receives feedback about the product and works on correcting bugs based on feedback provided by the customer.

REQUIREMENT ANALYSIS & SPECIFICATION

The software requirements are description of features and functionalities of the target system. Requirements convey the expectations of users from the software product. The requirements can be obvious or hidden, known or unknown, expected or unexpected from client's point of view.

Requirement Engineering

The process to gather the software requirements from client, analyze and document them is known as ***requirement engineering***.

The goal of requirement engineering is to develop and maintain sophisticated and descriptive 'System Requirements Specification' document.

Requirement Engineering Process

It is a four step process, which includes –

- Feasibility Study
- Requirement Gathering
- Software Requirement Specification
- Software Requirement

Validation Let us see the process

briefly -

Feasibility study

When the client approaches the organization for getting the desired product developed, it comes up with rough idea about what all functions the software must perform and which all features are expected from the software.

Referencing to this information, the analysts does a detailed study about whether the desired system and its functionality are feasible to develop.

This feasibility study is focused towards goal of the organization. This study analyzes whether the software product can be practically materialized in terms of implementation, contribution of project to organization, cost constraints and as per values and objectives of the organization. It

explores technical aspects of the project and product such as usability, maintainability, productivity and integration ability.

The output of this phase should be a feasibility study report that should contain adequate comments and recommendations for the management about whether or not the project should be undertaken.

Requirement Gathering

If the feasibility report is positive towards undertaking the project, next phase starts with gathering requirements from the user. Analysts and engineers communicate with the client and end-users to know their ideas on what the software should provide and which features they want the software to include.

Software Requirement Specification

SRS is a document created by the system analyst after the requirements are collected from various stakeholders.

SRS defines how the intended software will interact with hardware, external interfaces, speed of operation, response time of system, portability of software across various platforms, maintainability, speed of recovery after crashing, Security, Quality, Limitations etc.

The requirements received from client are written in natural language. It is the responsibility of system analyst to document the requirements in technical language so that they can be comprehended and useful by the software development team.

SRS should come up with following features:

- User requirements are expressed in natural language.
- Technical requirements are expressed in structured language, which is used inside the organization.
- Design description should be written in pseudo code.
- Format of Forms and GUI screen prints.
- Conditional and mathematical notations for DFDs etc.

Software Requirement Validation

After requirement specifications are developed, the requirements mentioned in this document are validated. User might ask for illegal, impractical solution or experts may interpret the requirements incorrectly. This results in huge increase in cost if not nipped in the bud.

Requirements can be checked against following conditions -

- If they can be practically implemented
- If they are valid and as per functionality and domain of software
- If there are any ambiguities
- If they are complete
- If they can be demonstrated

Requirement Elicitation Process

Requirement elicitation process can be depicted using the following diagram:



Fig 1: Requirement elicitation process

- **Requirements gathering** - The developers discuss with the client and the end users and know their expectations from the software.
- **Organizing Requirements** - The developers prioritize and arrange the requirements in order of importance, urgency and convenience.
- **Negotiation & discussion** - If the requirements are ambiguous or there are some conflicts in the requirements of various stakeholders, it is then negotiated and discussed with the stakeholders. Requirements may then be prioritized and reasonably compromised.

The requirements come from various stakeholders. To remove the ambiguity and conflicts, they are discussed for clarity and correctness. Unrealistic requirements are compromised reasonably.

- **Documentation** - All formal & informal, functional and non-functional requirements are documented and made available for next phase processing.

Requirement Elicitation Techniques

Requirements Elicitation is the process to find out the requirements for an intended software system by communicating with client, end users, system users and others who have a stake in the software system development.

There are various ways to discover requirements

Interviews

Interviews are strong medium to collect requirements. Organization may conduct several types of interviews such as:

- Structured (closed) interviews, where every single information to be gathered, is decided in advance, they follow the pattern and matter of discussion firmly.
- Non-structured (open) interviews, where information to gather is not decided in advance, more flexible and less biased.
- Oral interviews
- Written interviews
- One-to-one interviews which are held between two persons across the table.
- Group interviews which are held between groups of participants. They help to uncover any missing requirement as numerous people are involved.

Surveys

Organization may conduct surveys among various stakeholders by querying about their expectation and requirements from the upcoming system.

Questionnaires

A document with pre-defined set of objective questions and respective options is handed over to all stakeholders to answer, which are collected and compiled.

A shortcoming of this technique is, if an option for some issue is not mentioned in the questionnaire, the issue might be left unattended.

Task analysis

Team of engineers and developers may analyze the operation for which the new system is required. If the client already has some software to perform certain operation, it is studied and requirements of proposed system are collected.

Domain Analysis

Every software falls into some domain category. The expert people in the domain can be a great help to analyze general and specific requirements.

Brainstorming

An informal debate is held among various stakeholders and all their inputs are recorded for further requirements analysis.

Prototyping

Prototyping is building user interface without adding detail functionality for user to interpret the features of intended software product. It helps giving better idea of requirements. If there is no software installed at client's end for developer's reference and the client is not aware of its own requirements, the developer creates a prototype based on initially mentioned requirements. The prototype is shown to the client and the feedback is noted. The client feedback serves as an input for requirement gathering.

Observation

Team of experts visits the client's organization or workplace. They observe the actual working of the existing installed systems. They observe the workflow at client's end and how execution problems are dealt. The team itself draws some conclusions which aid to form requirements expected from the software.

SOFTWARE REQUIREMENT CHARACTERISTICS

Gathering software requirements is the foundation of the entire software development project. Hence they must be clear, correct and well-defined.

A complete Software Requirement Specifications must be:

- Clear
- Correct
- Consistent
- Coherent
- Comprehensible
- Modifiable
- Verifiable
- Prioritized
- Unambiguous
- Traceable
- Credible source

Software Requirements

We should try to understand what sort of requirements may arise in the requirement elicitation phase and what kinds of requirements are expected from the software system.

Broadly software requirements should be categorized in two categories:

Functional Requirements: Requirements, which are related to functional aspect of software fall into this category. They define functions and functionality within and from the software system.

EXAMPLES -

- Search option given to user to search from various invoices.
- User should be able to mail any report to management.
- Users can be divided into groups and groups can be given separate rights.
- Should comply business rules and administrative functions.
- Software is developed keeping downward compatibility intact.

Non-Functional Requirements: Requirements, which are not related to functional aspect of software, fall into this category. They are implicit or expected characteristics of software, which users make assumption of.

Non-functional requirements include -

- Security
- Logging
- Storage
- Configuration
- Performance
- Cost
- Interoperability
- Flexibility
- Disaster recovery
- Accessibility

Requirements are categorized logically as

- **Must Have:** Software cannot be said operational without them.
- **Should have:** Enhancing the functionality of software.
- **Could have:** Software can still properly function with these requirements.
- **Wish list:** These requirements do not map to any objectives of software.

While developing software, 'Must have' must be implemented, 'Should have' is a matter of debate with stakeholders and negotiation, whereas 'could have' and 'wish list' can be kept for software updates.

User Interface (UI) requirements

UI is an important part of any software or hardware or hybrid system. A software is widely accepted if it is -

- easy to operate
- quick in response
- effectively handling operational errors
- providing simple yet consistent user interface

User acceptance majorly depends upon how user can use the software. UI is the only way for users to perceive the system. A well performing software system must also be equipped with attractive, clear, consistent and responsive user interface. Otherwise the functionalities of software system cannot be used in convenient way. A system is said be good if it provides means to use it efficiently.

User interface requirements are briefly mentioned below -

- Content presentation
- Easy Navigation
- Simple interface
- Responsive
- Consistent UI elements
- Feedback mechanism
- Default settings
- Purposeful layout
- Strategically use of color and texture.
- Provide help information
- User centric approach
- Group based view settings.

Software System Analyst

System analyst in an IT organization is a person, who analyzes the requirement of proposed system and ensures that requirements are conceived and documented properly & correctly. Role of an analyst starts during Software Analysis Phase of SDLC. It is the responsibility of analyst to make sure that the developed software meets the requirements of the client.

System Analysts have the following responsibilities:

- Analyzing and understanding requirements of intended software
- Understanding how the project will contribute in the organization objectives
- Identify sources of requirement
- Validation of requirement
- Develop and implement requirement management plan
- Documentation of business, technical, process and product requirements

- Coordination with clients to prioritize requirements and remove any ambiguity
- Finalizing acceptance criteria with client and other stakeholders

Requirement elicitation techniques like FAST, QFD:

In requirement elicitation, the goal is to gather and understand the needs, goals, and expectations of stakeholders for a software project. Various techniques are employed to capture these requirements, and one such technique is FAST. Here's a breakdown of FAST and other common elicitation techniques:

FAST (Facilitated Application Specification Techniques)

FAST is an approach to requirement elicitation where the focus is on interactive, collaborative workshops. The technique helps to rapidly identify and specify requirements with the help of stakeholders.

Key Features of FAST:

Workshops: A series of facilitated workshops where stakeholders, including users, developers, and domain experts, come together to discuss requirements.

Focus on Active Participation: Stakeholders are actively involved in brainstorming, problem-solving, and discussing project needs, making the process highly collaborative.

User-Centric: FAST emphasizes the user's perspective and their needs, ensuring that the requirements reflect real-world use cases and scenarios.

Rapid Gathering: It's designed for fast-paced requirement gathering, helping avoid long, drawn-out processes.

Structured Approach: It uses specific methods like structured brainstorming or group discussions to gather clear and concise requirements.

Steps in the FAST Process:

Preliminary Preparation: Organize stakeholders, define objectives, and prepare material.

Facilitated Workshops: Conduct workshops to understand user needs and identify requirements. Techniques like brainstorming, role-playing, and storyboarding are used in this phase.

Documenting Requirements: The requirements are documented as they are discussed in

the workshop, ensuring everyone's input is captured.

Review and Prioritize: After gathering the requirements, review and prioritize them with stakeholders to ensure alignment with business goals.

Other Requirement Elicitation Techniques:

Interviews:

Description: One-on-one discussions between a business analyst (or system analyst) and stakeholders. Interviews are often structured (with predefined questions) or unstructured (open-ended discussions).

Advantages: Direct input from stakeholders, and detailed understanding can be gathered.

Disadvantages: Can be time-consuming, especially with a large number of stakeholders.

Surveys/Questionnaires:

Description: Written forms of questions that stakeholders fill out to gather specific information about requirements.

Advantages: Quick way to collect information from a large group of people, especially when stakeholders are geographically dispersed.

Disadvantages: Responses might be superficial or unclear, and follow-up clarification might be needed.

Focus Groups:

Description: A small group of stakeholders is brought together to discuss the software's requirements. This often includes users, managers, and subject matter experts.

Advantages: Interactive discussions can generate new ideas and insights.

Disadvantages: Might be influenced by dominant participants, and results may not be representative of the entire user base.

Observation:

Description: The analyst observes users in their work environment to understand the existing processes and identify potential requirements.

Advantages: Provides insight into how users perform their tasks and the challenges they face.

Disadvantages: Users may alter their behavior when they know they are being observed (Hawthorne effect).

Prototyping:

Description: Creating an early, simplified version of the system (a prototype) that stakeholders can interact with to clarify their requirements.

Advantages: Helps stakeholders visualize the end product and provides feedback on functionality.

Disadvantages: The prototype might lead stakeholders to expect a more refined product than what's possible at that stage.

Brainstorming:

Description: A group activity where participants generate as many ideas as possible about system requirements. The goal is to encourage creative thinking.

Advantages: Helps uncover requirements that may not have been considered initially.

Disadvantages: The ideas generated may not always be practical or feasible.

Use Cases/Scenarios:

Description: Descriptions of how users will interact with the system in specific scenarios. This technique focuses on user behavior and system responses.

Advantages: Helps clarify system requirements by focusing on user interactions.

Disadvantages: It may be difficult to capture all possible interactions, and scenarios may not cover edge cases.

JAD (Joint Application Development):

Description: Facilitated meetings where users, analysts, and IT staff collaborate to define system requirements. It is a structured, intensive process designed to collect requirements quickly.

Advantages: Reduces time spent on gathering requirements, encourages active participation,

and ensures consensus.

Disadvantages: Can be time-consuming for participants, especially if there are many stakeholders.

Quality Function Deployment (QFD)

QFD is a method that helps prioritize customer requirements and translate them into engineering characteristics. It often involves creating a House of Quality (HoQ), which is a matrix used to correlate customer needs with engineering solutions.

Key Features of QFD:

Focus on Customer Needs: The primary goal of QFD is to capture the voice of the customer (VOC). It ensures that customer requirements are clearly understood and reflected in the final product.

Cross-Functional Collaboration: QFD involves a collaborative approach, bringing together cross-functional teams (e.g., marketing, design, engineering, production) to ensure that all perspectives are considered in the requirement gathering process.

Structured Approach: QFD uses structured matrices, particularly the **House of Quality (HoQ)**, to visually represent the relationship between customer needs and technical specifications.

Prioritization: QFD helps prioritize requirements based on customer importance and technical feasibility, ensuring the product meets both user expectations and engineering constraints.

How QFD Works in Requirement Elicitation:

Identify Customer Requirements (What's):

The first step in QFD is to gather and identify **customer needs and expectations**. These could come from customer surveys, focus groups, or direct interviews.

The **"What's"** in the QFD process represent these customer requirements.

Identify Engineering Characteristics (How's):

After gathering customer requirements, the next step is to identify the technical specifications or engineering characteristics required to meet these needs. These are referred

to as the "**How's**".

For example, if a customer needs a fast-loading website, the "How" might be to use specific coding practices to optimize load times.

Create the House of Quality (HoQ):

The **House of Quality** is a matrix that helps to map customer requirements (What's) against engineering characteristics (How's). The rows represent customer requirements, and the columns represent the technical requirements.

The matrix allows the team to evaluate how well each engineering characteristic addresses a customer need and identify which ones are most critical to the success of the product.

Prioritize Requirements:

QFD helps to prioritize the customer requirements and technical features by assigning importance values or weights to each one. This prioritization helps ensure that the most critical needs are addressed first in the development process.

Different methods like rating scales or competitive analysis can be used to assign importance values.

Refine the Design:

As the product is developed, the HoQ matrix is continually refined to ensure that engineering changes continue to meet the evolving customer requirements.

Advantages of QFD:

Customer-Centric: QFD ensures that the development process remains focused on delivering value to the customer. By translating customer needs directly into engineering specifications, it increases the likelihood of customer satisfaction.

Prevents Miscommunication: By using a clear and structured approach, QFD helps prevent misunderstandings between different departments (e.g., design, marketing, engineering).

Reduces Development Costs: By identifying critical requirements early in the process and focusing on them, QFD helps avoid costly changes later in the development cycle.

Encourages Cross-Functional Collaboration: QFD promotes collaboration between departments (marketing, engineering, production), which leads to more integrated product development.

SOFTWARE ANALYSIS & DESIGN TOOLS

Software analysis and design includes all activities, which help the transformation of requirement specification into implementation. Requirement specifications specify all functional and non-functional expectations from the software. These requirement specifications come in the shape of human readable and understandable documents, to which a computer has nothing to do.

Software analysis and design is the intermediate stage, which helps human-readable requirements to be transformed into actual code.

Let us see few analysis and design tools used by software designers:

Data Flow Diagram

Data flow diagram is a graphical representation of data flow in an information system. It is capable of depicting incoming data flow, outgoing data flow and stored data. The DFD does not mention anything about how data flows through the system.

There is a prominent difference between DFD and Flowchart. The flowchart depicts flow of control in program modules. DFDs depict flow of data in the system at various levels. DFD does not contain any control or branch elements.

Types of DFD

Data Flow Diagrams are either Logical or Physical.

- **Logical DFD** - This type of DFD concentrates on the system process and flow of data in the system. For example in a Banking software system, how data is moved between different entities.
- **Physical DFD** - This type of DFD shows how the data flow is actually implemented in the system. It is more specific and close to the implementation.

DFD Components

DFD can represent Source, destination, storage and flow of data using the following set of components -



Fig 1: DFD Components

- **Entities** - Entities are source and destination of information data. Entities are represented by rectangles with their respective names.
- **Process** - Activities and action taken on the data are represented by Circle or Round- edged rectangles.
- **Data Storage** - There are two variants of data storage - it can either be represented as a rectangle with absence of both smaller sides or as an open-sided rectangle with only one side missing.
- **Data Flow** - Movement of data is shown by pointed arrows. Data movement is shown from the base of arrow as its source towards head of the arrow as destination.

Levels of DFD

- **Level 0** - Highest abstraction level DFD is known as Level 0 DFD, which depicts the entire information system as one diagram concealing all the underlying details. Level 0 DFDs are also known as context level DFDs.

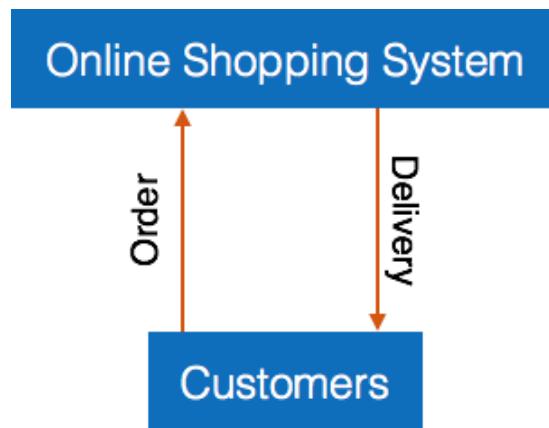


Fig 1: Level 0 DFD of Online Shopping System

- **Level 1** - The Level 0 DFD is broken down into more specific, Level 1 DFD. Level 1 DFD depicts basic modules in the system and flow of data among various modules. Level 1 DFD also mentions basic processes and sources of information.

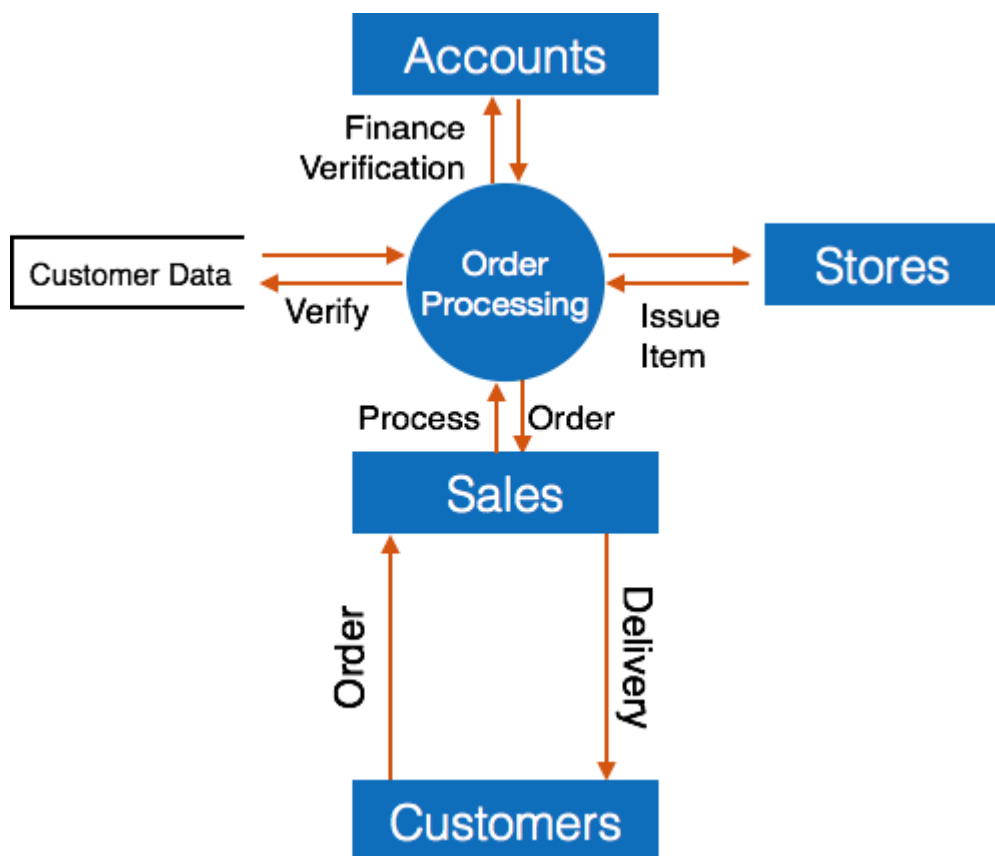


Fig 2: Level 1 DFD of Online Shopping System

- **Level 2** - At this level, DFD shows how data flows inside the modules mentioned in Level 1.

Higher level DFDs can be transformed into more specific lower level DFDs with deeper level of understanding unless the desired level of specification is achieved.

STRUCTURE CHART

Structure chart is a chart derived from Data Flow Diagram. It represents the system in more detail than DFD. It breaks down the entire system into lowest functional modules, describes functions and sub-functions of each module of the system to a greater detail than DFD.

Structure chart represents hierarchical structure of modules. At each layer a specific task is performed.

Here are the symbols used in construction of structure charts -

- **Module** - It represents process or subroutine or task. A control module branches to more than one sub-module. Library Modules are re-usable and invocable from any module.

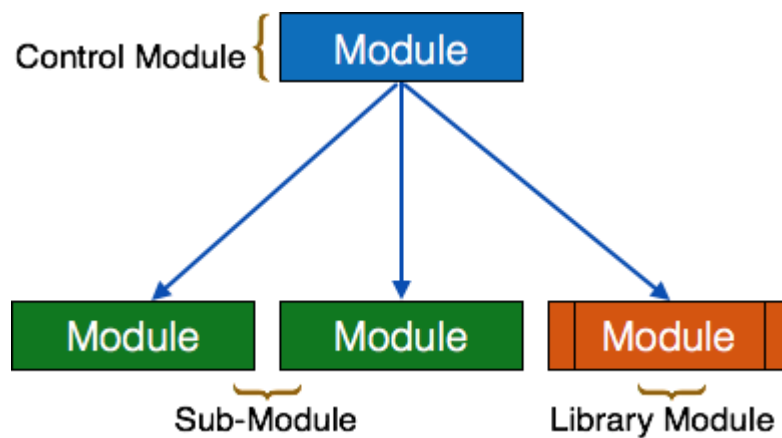


Fig 3: Modules in a structure chart

- **Condition** - It is represented by small diamond at the base of module. It depicts that control module can select any of sub-routine based on some condition.

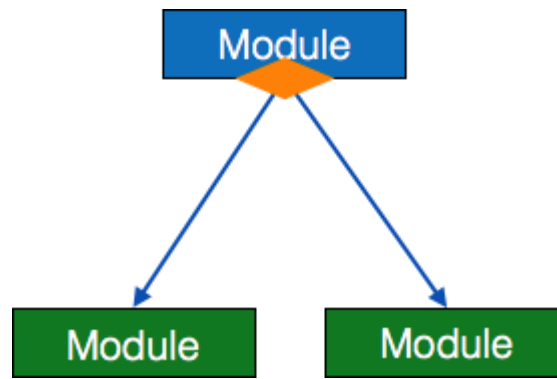


Fig 4: Condition in a structure chart

- **Jump** - An arrow is shown pointing inside the module to depict that the control will jump in the middle of the sub module.

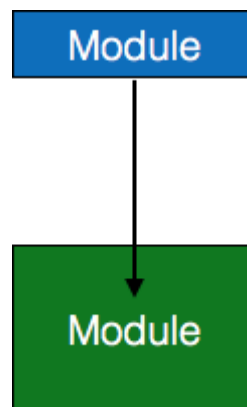


Fig 5: Jump in a structure chart

- **Loop** - A curved arrow represents loop in the module. All sub-modules covered by loop repeat execution of module.

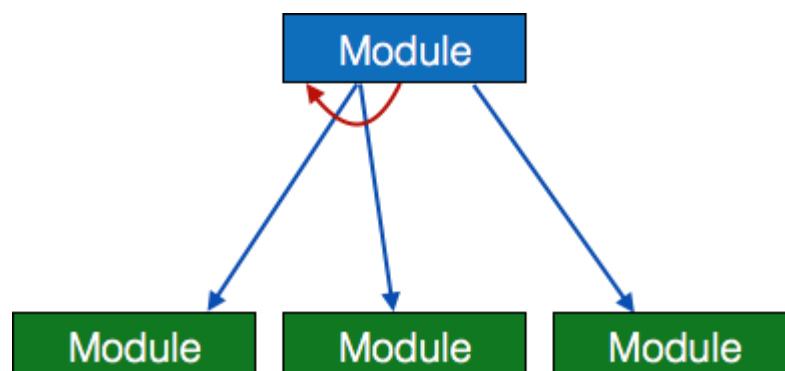


Fig 6: Loop in a structure chart

- **Data flow** - A directed arrow with empty circle at the end represents data flow.

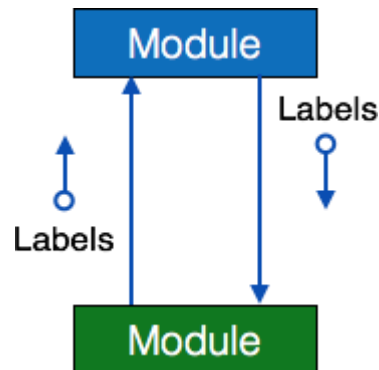


Fig 7: Data Flow in a structure chart

- **Control flow** - A directed arrow with filled circle at the end represents control flow.

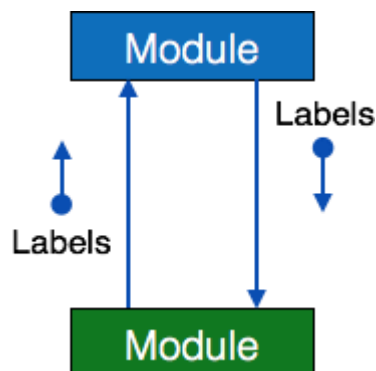


Fig 7: Control Flow in a structure chart

Structured English

Most programmers are unaware of the large picture of software so they only rely on what their managers tell them to do. It is the responsibility of higher software management to provide accurate information to the programmers to develop accurate yet fast code.

Other forms of methods, which use graphs or diagrams, may be sometimes interpreted differently by different people.

Hence, analysts and designers of the software come up with tools such as Structured English. It is nothing but the description of what is required to code and how to code it. Structured English helps the programmer to write error-free code.

Other forms of methods, which use graphs or diagrams, may be sometimes interpreted differently by different people. Here, both Structured English and Pseudo-Code try to mitigate that understanding gap.

Structured English uses plain English words in structured programming paradigm. It is not the ultimate code but a kind of description of what is required to code and how to code it. The following are some tokens of structured programming.

IF-THEN-ELSE,

DO-WHILE-

UNTIL

Analyst uses the same variable and data name, which are stored in Data Dictionary, making it much simpler to write and understand the code.

Example

We take the same example of Customer Authentication in the online shopping environment. This procedure to authenticate customer can be written in Structured English as:

```
Enter Customer_Name  
SEEK Customer_Name in Customer_Name_DB  
file IF Customer_Name found THEN  
    Call procedure USER_PASSWORD_AUTHENTICATE()  
ELSE  
    PRINT error message  
Call procedure NEW_CUSTOMER_REQUEST()  
ENDIF
```

The code written in Structured English is more like day-to-day spoken English. It cannot be implemented directly as a code of software. Structured English is independent of programming language.

Pseudo-Code

Pseudo code is written more close to programming language. It may be considered as augmented programming language, full of comments and descriptions.

Pseudo code avoids variable declaration but they are written using some actual programming language's constructs, like C, Fortran, Pascal etc.

Pseudo code contains more programming details than Structured English. It provides a method to perform the task, as if a computer is executing the code.

Example

Program to print Fibonacci up to n numbers.

```
void function  
Fibonacci Get value of  
n;  
Set value of a  
to 1; Set value  
of b to 1;  
Initialize I to 0  
for (i=0; i< n; i++)  
{  
    if a greater than b  
    {  
        Increase b by a;  
        print b;  
    }  
    else if b greater than a  
    {  
        increase a by b;
```

```

        print a;
    }
}

```

DECISION TABLES

A Decision table represents conditions and the respective actions to be taken to address them, in a structured tabular format.

It is a powerful tool to debug and prevent errors. It helps group similar information into a single table and then by combining tables it delivers easy and convenient decision-making.

Creating Decision Table

To create the decision table, the developer must follow basic four steps:

- Identify all possible conditions to be addressed
- Determine actions for all identified conditions
- Create Maximum possible rules
- Define action for each rule

Decision Tables should be verified by end-users and can lately be simplified by eliminating duplicate rules and actions.

Example

Let us take a simple example of day-to-day problem with our Internet connectivity. We begin by identifying all problems that can arise while starting the internet and their respective possible solutions.

We list all possible problems under column conditions and the prospective actions under column actions.

	Conditions/Actions	Rules							
Conditions	Shows Connected	N	N	N	N	Y	Y	Y	Y
	Ping is Working	N	N	Y	Y	N	N	Y	Y
	Opens Website	Y	N	Y	N	Y	N	Y	N

Actions	Check network cable	X							
	Check internet router	X				X	X	X	
	Restart Web Browser							X	
	Contact Service provider		X	X	X	X	X	X	

Table 1: Decision Table – In-house Internet Troubleshooting

Entity-Relationship Model

Entity-Relationship model is a type of database model based on the notion of real world entities and relationship among them. We can map real world scenario onto ER database model. ER Model creates a set of entities with their attributes, a set of constraints and relation among them.

ER Model is best used for the conceptual design of database. ER Model can be represented as follows:

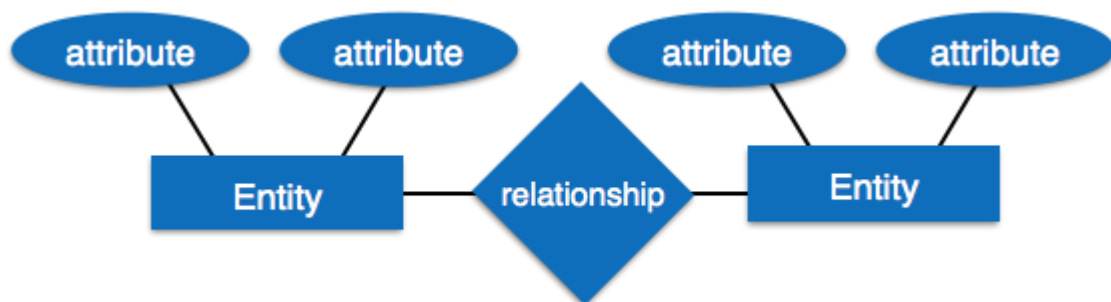


Fig 8: ER Model

- **Entity** - An entity in ER Model is a real world being, which has some properties called **attributes**. Every attribute is defined by its corresponding set of values, called **domain**.

For example, consider a school database. Here, a student is an entity. Student has various attributes like name, id, age and class etc.

- **Relationship** - The logical association among entities is called **relationship**. Relationships are mapped with entities in various ways. Mapping cardinalities define the number of associations between two entities.

Mapping cardinalities:

- one to one
- one to many
- many to one
- many to many

Data Dictionary

Data dictionary is the centralized collection of information about data. It stores meaning and origin of data, its relationship with other data, data format for usage etc. Data dictionary has rigorous definitions of all names in order to facilitate user and software designers.

Data dictionary is often referenced as meta-data (data about data) repository. It is created along with DFD (Data Flow Diagram) model of software program and is expected to be updated whenever DFD is changed or updated.

Requirement of Data Dictionary

The data is referenced via data dictionary while designing and implementing software. Data dictionary removes any chances of ambiguity. It helps keeping work of programmers and designers synchronized while using same object reference everywhere in the program.

Data dictionary provides a way of documentation for the complete database system in one place. Validation of DFD is carried out using data dictionary.

Contents

Data dictionary should contain information about the following

- Data Flow
- Data Structure
- Data Elements
- Data Stores
- Data Processing

Data Flow is described by means of DFDs as studied earlier and represented in algebraic form as described.

=	Composed of
{ }	Repetition
()	Optional
+	And
[/]	Or

Example

Address = House No + (Street / Area) + City + State

Course ID = Course Number + Course Name + Course Level + Course Grades

Data Elements

Data elements consist of Name and descriptions of Data and Control Items, Internal or External data stores etc. with the following details:

- Primary Name
- Secondary Name (Alias)
- Use-case (How and where to use)
- Content Description (Notation etc.)
- Supplementary Information (preset values, constraints

etc.) Data Store

It stores the information from where the data enters into the system and exists out of the system. The Data Store may include -

- **Files**
 - Internal to software.
 - External to software but on the same machine.
 - External to software and system, located on different machine.
- **Tables**
 - Naming convention
 - Indexing

property Data Processing

There are two types of Data Processing:

- **Logical:** As user sees it
- **Physical:** As software sees it

Library Management System:

Library Management System Overview

A Library Management System (LMS) is a software application designed to manage the operations of a library. This includes managing books, users (students, faculty, etc.), borrowing and returning books, and keeping track of transactions.

Entities and Functionalities:

Entities: Books, Users (Students, Faculty), Staff, Transactions (Issue, Return).

Functions:

Add/Remove books

Issue/Return books

Maintain user accounts

Track overdue books and fines

Search book

Entity-Relationship (E-R) Diagram

The E-R diagram describes the structure of the library database in terms of entities, attributes, and relationships. Below is a textual representation:

Entities:

Book: BookID, Title, Author, Publisher, Year

User: UserID, Name, Email, Phone

Transaction: TransactionID, IssueDate, ReturnDate, Fine

Staff: StaffID, Name, Role

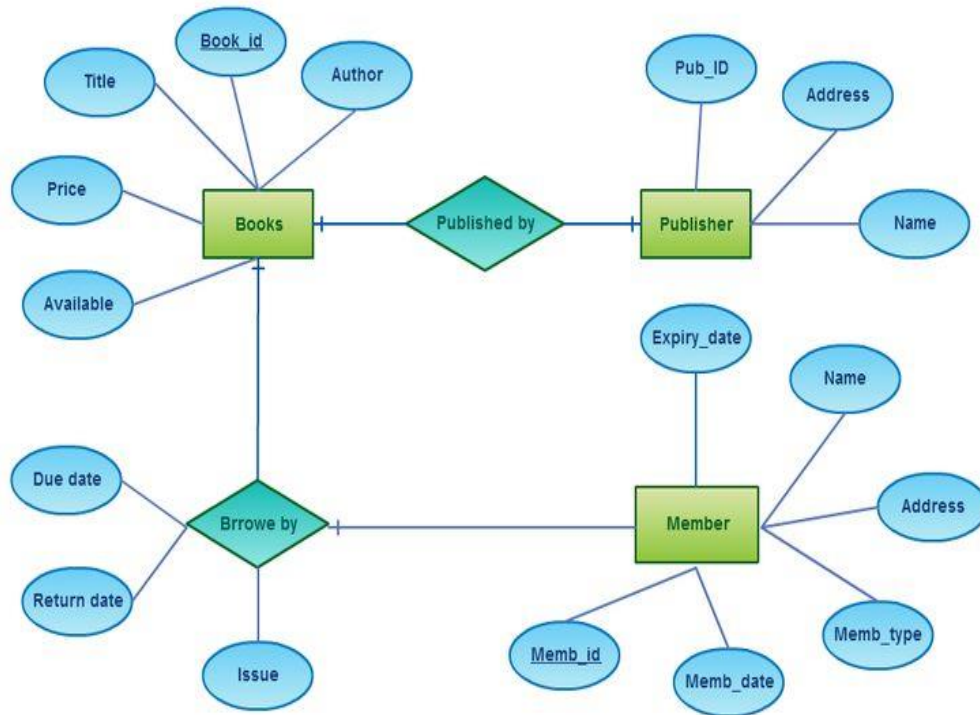
Relationships:

A User can have many Transactions (one-to-many).

A Transaction involves one Book (many-to-one).

A Staff handles many Transactions (one-to-many).

E-R Diagram of Library Management System

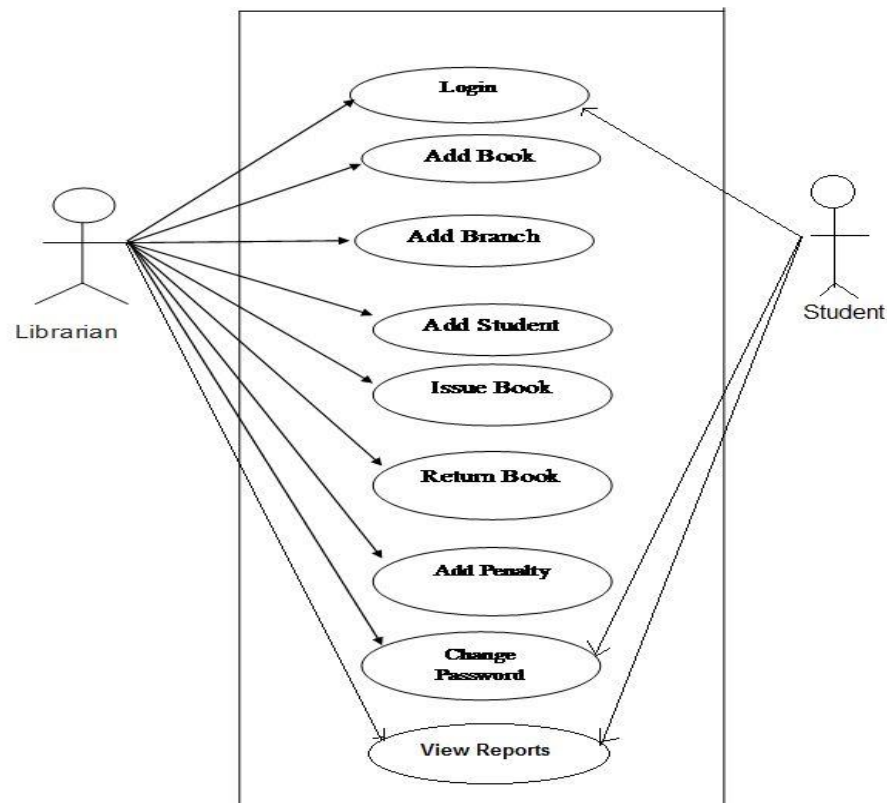


2. Use Case Diagram

A Use Case diagram shows the interaction between the system and external users. In this case, the external actors are **Users** and **Staff**. Here are the typical use cases for the Library Management System:

Use Cases:

- **User:**
 - Search Books
 - Borrow Book
 - Return Book
 - Pay Fine
- **Staff:**
 - Add/Remove Books
 - Issue Book
 - Receive Returned Book
 - Manage User Account
 - Generate Report (e.g., overdue books)



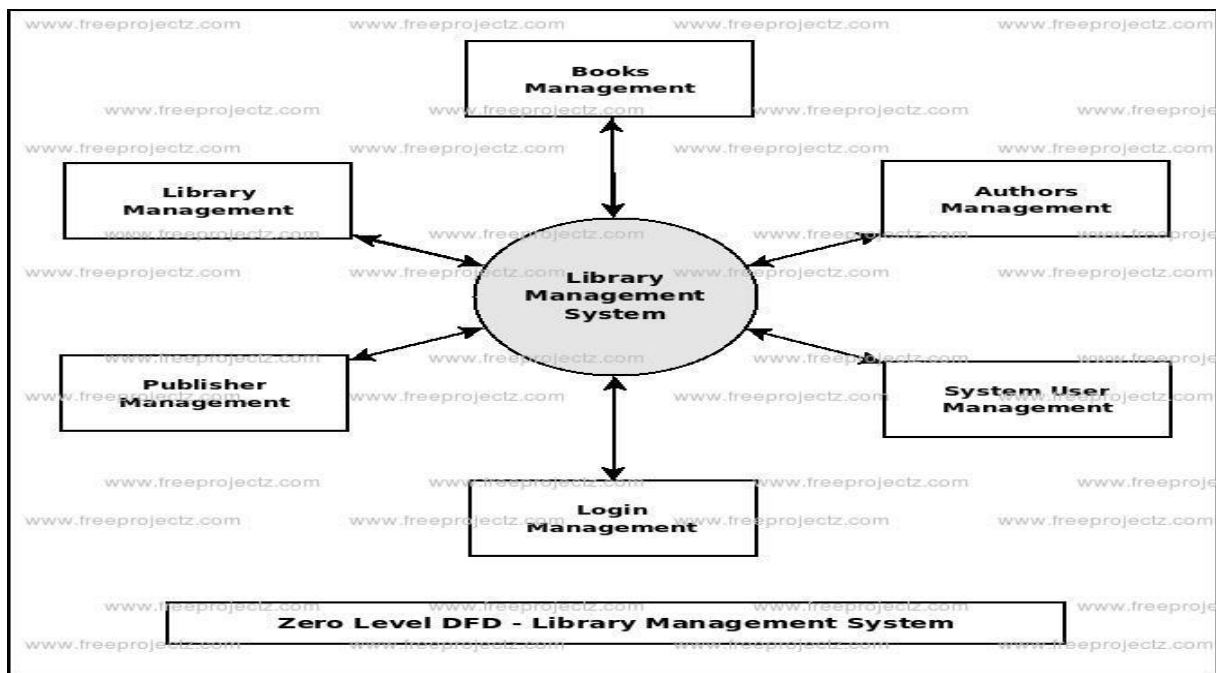
Two-Level Data Flow Diagram (DFD)

A **Data Flow Diagram (DFD)** shows how data moves within the system. A **two-level DFD** typically consists of:

1. **Level 0 DFD (Context Diagram):** Represents the system as a single process with external actors (users and staff).
2. **Level 1 DFD:** Breaks the system down into major sub-processes.

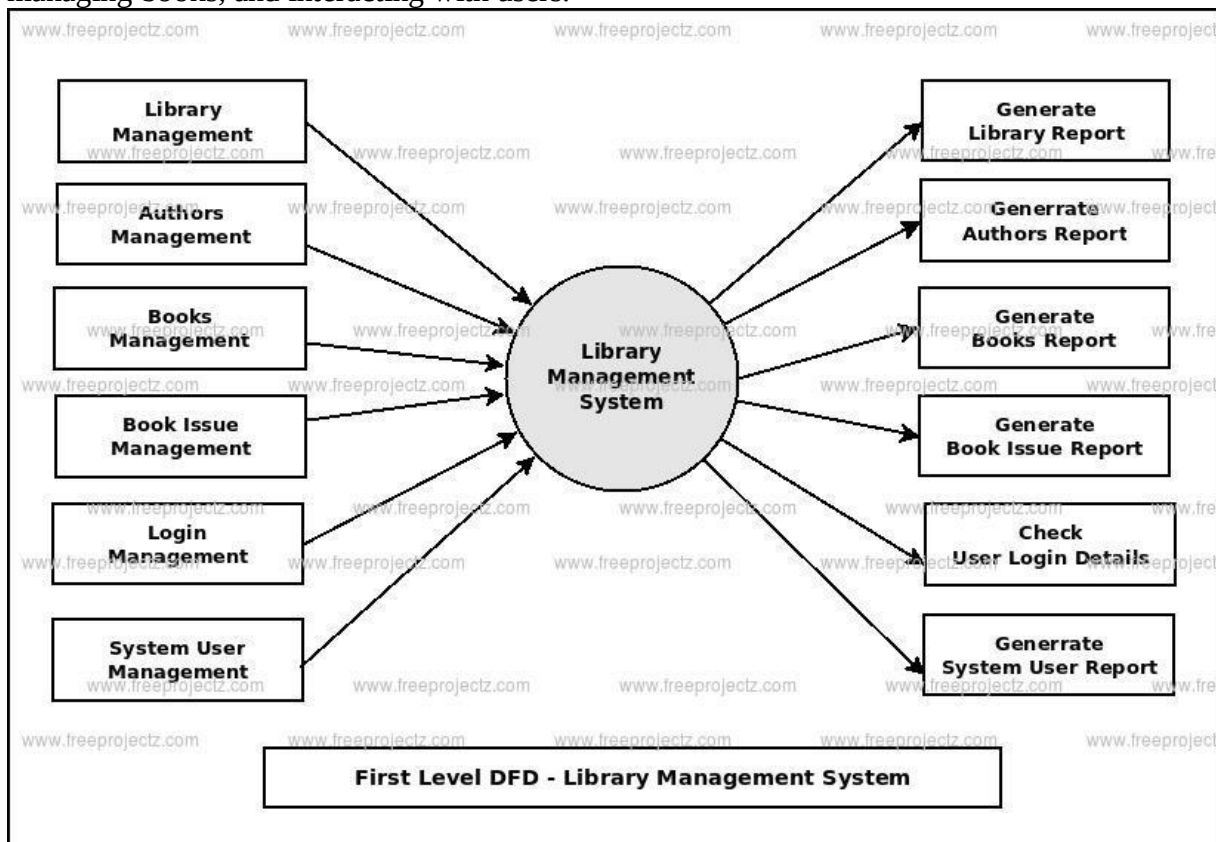
Level 0 DFD (Context Diagram):

This is a high-level DFD that represents the entire system as a single process interacting with external entities.



Level 1 DFD:

This diagram provides more detail about the system's internal processes, such as handling transactions, managing books, and interacting with users.



Requirements documentation: SRS, Characteristics & organization of SRS

Software Requirements Specification (SRS)

A **Software Requirements Specification (SRS)** is a detailed document that outlines the functional and non-functional requirements of a software system. It serves as a guide for both developers and stakeholders, helping ensure that the software system will meet the users' needs and expectations. The SRS is typically created during the early stages of software development and serves as a contract between the client and the development team.

Key Components of an SRS:

An SRS is structured to ensure clarity, consistency, and completeness. The typical components of an SRS document include:

1. Introduction

- **Purpose:** Describes the purpose of the SRS, the software product, and the audience for the document.
- **Scope:** Defines the boundaries of the software system, including what is included and excluded from the system.
- **Definitions, Acronyms, and Abbreviations:** Lists any technical terms, acronyms, or abbreviations used throughout the document.
- **References:** Lists other documents that are relevant to the SRS, such as system specifications or regulatory standards.
- **Overview:** Provides an overview of the rest of the document.

2. Overall Description

- **Product Perspective:** Describes the software's position within a larger system (if applicable). For example, how the software will interact with other systems or external devices.
- **Product Features:** Summarizes the key features of the product at a high level, including its objectives and goals.
- **User Characteristics:** Describes the user groups, including their skill level, technical expertise, and needs.
- **Constraints:** Lists any limitations or restrictions (e.g., technical, legal, or business-related) that the software must adhere to.
- **Assumptions and Dependencies:** States any assumptions made during the development and any external dependencies that the system has (e.g., external hardware, third-party software).

3. System Features

- **Functional Requirements:** This section contains a detailed description of the system's behavior and what functions the system must perform. Each feature is typically described as:
 - **Description:** A high-level explanation of the feature.
 - **Stimulus/Response Sequences:** How the system should respond to particular inputs or events.
 - **Functional Requirements:** Detailed requirements for the feature, which specify the exact functionality the system must exhibit.

4. External Interface Requirements

- **User Interfaces:** Describes the interface through which users will interact with the system. This can include mockups, user interface guidelines, and constraints.
- **Hardware Interfaces:** Details the interaction between the software and external hardware components.
- **Software Interfaces:** Describes the communication and integration between the system and other software or services.
- **Communication Interfaces:** Specifies the protocols, communication methods, and formats for data exchange between the system and external entities.

5. System Attributes (Non-Functional Requirements)

- **Performance Requirements:** Defines system performance criteria such as response time, throughput, and availability.
- **Safety Requirements:** Specifies any safety-related requirements or standards the software must comply with.
- **Security Requirements:** Details security specifications such as authentication, authorization, data encryption, and audit trails.
- **Maintainability:** Describes how easy it should be to maintain and update the software over time.
- **Reliability:** Defines how dependable the system needs to be in terms of uptime and error handling.
- **Scalability:** Outlines how the system should be able to grow in terms of usage or size.
- **Usability:** Defines how easy it should be for users to interact with the system, including UI design and user experience considerations.

6. Other Requirements

- This section is used for any additional requirements that don't fit neatly into other categories. It might include legal or regulatory requirements, environmental constraints, or specific compliance needs.

Benefits of an SRS:

1. **Clear Communication:** Provides a clear understanding of system requirements between stakeholders (such as customers, developers, testers, and project managers). This minimizes misunderstandings and miscommunications.
2. **Consistency and Accuracy:** Helps to ensure the software meets the client's needs and expectations by setting out precise and unambiguous requirements.
3. **A Reference for Validation:** The SRS acts as a reference for system testing. Developers and testers can validate the software against the requirements to ensure it functions correctly.
4. **Risk Management:** By detailing requirements in advance, the SRS helps identify and address potential risks early in the development process.
5. **Foundation for Design and Development:** The SRS serves as the foundation for the subsequent stages of development, including architectural design, coding, and system testing. It ensures that all parties involved understand the goals and features of the software.

Best Practices for Writing an SRS:

1. **Clarity and Precision:** The SRS should be written clearly and concisely. Ambiguities should be avoided to ensure that every requirement is understandable.
2. **Use of Standard Terminology:** Using standardized terminology (such as "shall" for mandatory requirements) helps avoid confusion and misinterpretation.
3. **Modularity and Organization:** Break the SRS into clear, logical sections, each dealing with a different aspect of the software, to make it easy to reference and navigate.
4. **Validation and Verification:** Continuously validate and verify the SRS by reviewing it with stakeholders to ensure that it accurately represents the needs and expectations of the users.
5. **Consistency with Other Documents:** Ensure that the SRS aligns with other project documents, such as project plans, business requirements, and regulatory guidelines.

The Software Requirements Specification (SRS) document serves as a detailed blueprint for the software system to be developed. It provides clear, concise, and unambiguous descriptions of the software's functional and non-functional requirements, guiding both developers and stakeholders throughout the software development lifecycle.

Key Characteristics of a Good SRS:

1. **Correctness:**
 - **Definition:** The SRS should accurately represent the needs of the stakeholders and the system's requirements.
 - **Explanation:** Every requirement should correctly reflect what is needed for the system to fulfill its intended purpose. Misunderstandings or incorrect assumptions during requirements gathering can lead to a product that doesn't meet user expectations.
2. **Unambiguity:**
 - **Definition:** Requirements in the SRS should be clear and leave no room for multiple interpretations.
 - **Explanation:** Ambiguity in requirements can lead to confusion during the development process. The use of precise language, standard terminology, and well-defined terms is essential to ensure there is only one interpretation for each requirement.
3. **Completeness:**
 - **Definition:** The SRS should cover all possible scenarios and aspects of the system's functionality and behavior.
 - **Explanation:** A complete SRS includes all functional, non-functional, and interface requirements, as well as constraints and assumptions. If a requirement is missing or unclear, it can result in features being left out of the final system or functionality being misunderstood.
4. **Consistency:**
 - **Definition:** The SRS should not contain contradictory requirements.
 - **Explanation:** Conflicting requirements or contradictory descriptions can cause confusion during the development process. It's essential that all requirements align and are consistent with each other across different sections of the document.
5. **Verifiability:**
 - **Definition:** Every requirement should be testable or verifiable.

- **Explanation:** Each requirement should be measurable so that it can be verified through testing, inspection, or demonstration. Non-verifiable requirements make it difficult to assess whether the system meets the specified criteria.
6. **Modifiability:**
- **Definition:** The SRS should be easy to update and modify as requirements evolve during the project.
 - **Explanation:** Requirements can change throughout the development lifecycle due to new insights, stakeholder feedback, or changes in business objectives. The SRS should be structured in a way that allows for easy modification, without causing confusion or inconsistency.
7. **Traceability:**
- **Definition:** Each requirement should be uniquely identifiable and traceable throughout the development process.
 - **Explanation:** Traceability allows teams to trace each requirement back to its source (e.g., customer needs, business goals) and to ensure that the final product meets those requirements. It also helps manage changes by linking them to specific parts of the system.
8. **Understandability:**
- **Definition:** The SRS should be written in a clear and understandable manner for all stakeholders.
 - **Explanation:** The document should be readable by both technical and non-technical stakeholders, including developers, testers, project managers, and business users. The language should be simple, straightforward, and free from technical jargon when unnecessary.
9. **Feasibility:**
- **Definition:** The requirements in the SRS should be realistic and achievable within the project's constraints.
 - **Explanation:** A good SRS ensures that all requirements can be implemented within the available resources, technology, and timeframes. Unrealistic or impossible requirements can lead to project failure or delays.
10. **Prioritization:**
- **Definition:** Requirements in the SRS should be ranked by importance.
 - **Explanation:** Not all requirements are equally important. The SRS should reflect the priority of each requirement, allowing development teams to focus on the most critical features first. This is especially important when dealing with limited resources or tight deadlines.
11. **Traceability:**
- **Definition:** The SRS should provide a way to trace requirements to their source and ensure they are implemented correctly.
 - **Explanation:** Traceability is critical for managing changes during the project. If a requirement changes, the impact on other parts of the system can be analyzed easily by tracing dependencies.
12. **Non-ambiguity in Language:**
- **Definition:** The language used in the SRS must be precise, avoiding vague terms or unclear definitions.
 - **Explanation:** Terms such as "fast," "easy," or "sufficient" can lead to different interpretations. Clear definitions and standardized terms must be used to ensure that everyone involved understands the requirements in the same way.

Note:

The SRS is a critical document that defines the foundation of the entire software development process. To be effective, an SRS must be **correct, unambiguous, complete, consistent, verifiable, modifiable, traceable, understandable, feasible, and prioritized**. These characteristics ensure that the requirements gathered are of high quality, making the software development process smoother, reducing the risk of errors, and leading to a product that aligns well with the user's needs and expectations.

Organisation of SRS:

The **organization of a Software Requirements Specification (SRS)** document follows a structured layout that ensures all relevant information about the software system is clearly and comprehensively captured. The typical structure is designed to cover all aspects of the software, including functionality, performance, and interfaces, and to serve as a detailed guide for the development team. The **IEEE 830-1998** standard for SRS is commonly used as a template for organizing SRS documents, though organizations may adapt it to meet specific needs.

Typical Organization of an SRS Document

1. Introduction

This section provides a high-level overview of the document, including the purpose, scope, and intended audience.

- **1.1 Purpose:** Describes the purpose of the SRS document and the software system it specifies. It explains what the document aims to achieve and who the intended audience is (e.g., developers, project managers, users).
- **1.2 Scope:** Defines the boundaries of the software system, outlining what is included and excluded in the system's functionality. It also mentions any key limitations or constraints.
- **1.3 Definitions, Acronyms, and Abbreviations:** Provides a list of all relevant terms, acronyms, and abbreviations used in the document. This ensures that everyone has a common understanding of technical terms and language.
- **1.4 References:** Lists documents or resources referenced within the SRS. These could include other specification documents, standards, user manuals, or related research.
- **1.5 Overview:** Summarizes the structure of the rest of the SRS document. It provides a brief outline of the upcoming sections to give the reader a clear idea of what to expect.

2. Overall Description

The overall description provides context about the system, including its goals, user requirements, and dependencies.

- **2.1 Product Perspective:** Describes the software's context within the larger system or its relationship to other systems. This could include information about how it integrates with existing systems, databases, or hardware.
- **2.2 Product Features:** Provides a high-level description of the system's main features. This section outlines the key functionalities the system will provide, but without going into deep technical details.
- **2.3 User Classes and Characteristics:** Identifies the different user groups who will interact with the system and their characteristics (e.g., skill level, technical background, role in the organization).
- **2.4 Operating Environment:** Specifies the hardware, software, and network environment in which the system will operate. This might include operating system requirements, required platforms, or database systems.
- **2.5 Design and Implementation Constraints:** Lists any limitations that will affect the design or implementation of the system, such as hardware constraints, regulatory requirements, or technical restrictions.
- **2.6 Assumptions and Dependencies:** Describes any assumptions made during the development process (e.g., reliance on specific third-party software or technologies) and external factors that could impact the project's success.

3. System Features

This section provides detailed functional requirements for each feature or capability of the system. Each feature is broken down into the following components:

- **3.x Feature Name:** Each feature is given a unique identifier or title (e.g., "3.1 User Login" or "3.2 Search Functionality").
- **Description:** A high-level description of what the feature does and how it supports the overall system's goals.
- **Stimulus/Response Sequences:** Explains the interaction flow for the feature. What inputs trigger specific outputs, and how does the system respond to different stimuli?
- **Functional Requirements:** A more detailed, specific set of requirements that describe the necessary behavior of the system for the feature. Each requirement should be clear and verifiable, specifying the expected inputs, processes, and outputs.

4. External Interface Requirements

This section focuses on how the system will interact with other systems, hardware, or users. It defines the system's interfaces to ensure proper integration.

- **4.1 User Interfaces:** Describes the design of the user interface, including layout, navigation, and interactions. This may include mockups, wireframes, or user experience guidelines.
- **4.2 Hardware Interfaces:** Specifies how the system will interact with hardware, such as sensors, actuators, or physical devices.
- **4.3 Software Interfaces:** Details how the system will integrate with other software systems or services, including APIs, data formats, and protocols.

- **4.4 Communication Interfaces:** Describes the methods of communication, such as network protocols, data transmission methods, and encryption standards, used for system-to-system communication.

5. System Attributes (Non-Functional Requirements)

This section defines the **non-functional requirements** that are critical for system performance, reliability, and security. These requirements focus on how well the system performs, rather than what it does.

- **5.1 Performance Requirements:** Specifies the system's required response time, throughput, latency, or other performance metrics. This may include how fast the system needs to process data or respond to user requests.
- **5.2 Safety Requirements:** Details any safety protocols, features, or concerns that must be considered, particularly in systems where user safety or environmental safety is a concern (e.g., medical systems, automotive systems).
- **5.3 Security Requirements:** Defines the security measures the system must include, such as data encryption, authentication, user authorization, and protection against threats.
- **5.4 Reliability:** Describes the expected uptime, error handling, and failure recovery mechanisms for the system. It defines how the system should behave in the event of a failure.
- **5.5 Availability:** Specifies the availability requirements, such as the desired level of uptime or the system's operational hours.
- **5.6 Maintainability:** Defines how easy it should be to maintain and upgrade the system, including the ability to apply patches, updates, and changes without causing significant downtime.
- **5.7 Scalability:** Describes the system's ability to handle increased workload or user base. This could include requirements for system expansion or upgrades as demand grows.
- **5.8 Usability:** Specifies how user-friendly and intuitive the system should be. This could include user interface design principles and how the system should be optimized for ease of use.

6. Other Requirements

This section includes any additional requirements that do not fit into the categories above but are still important to the project.

- **6.1 Legal and Regulatory Requirements:** Lists any legal, regulatory, or compliance requirements that the system must meet (e.g., GDPR, HIPAA).
- **6.2 Environmental Requirements:** Describes any environmental constraints or conditions that the system must operate within (e.g., temperature ranges, humidity conditions).
- **6.3 Data Requirements:** Specifies the system's requirements for data storage, data formats, backup, and recovery.

- **6.4 Documentation Requirements:** Details any documentation that must be produced, such as user manuals, API documentation, or developer guides.

Conclusion

The organization of an **SRS document** follows a structured approach to ensure that all aspects of the system are thoroughly documented, clearly specified, and well-understood by all stakeholders. By adhering to this structure, the development team can ensure that the software is built according to the client's expectations and can be properly tested and maintained in the future.

An effective SRS ensures that the system is built right the first time, reducing the risk of miscommunication, errors, and changes during the development lifecycle.